Boring but important disclaimers:

- If you are not getting this from the GitHub repository or the associated Canvas page (e.g. CourseHero, Chegg etc.), you are probably getting the substandard version of these slides Don't pay money for those, because you can get the most updated version for free at

  `https://github.com/julianmak/OCES5303_ML_ocean`

  The repository principally contains the compiled products rather than the source for size reasons.
- Associated Python code (as Jupyter notebooks mostly) will be held on the same repository. The source data however might be big, so I am going to be naughty and possibly just refer you to where you might get the data if that is the case (e.g. JRA-55 data). I know I should make properly reproducible binders etc., but I didn't...
- I do not claim the compiled products and/or code are completely mistake free (e.g. I know I don't write Pythonic code). Use the material however you like, but use it at your own risk.
- As said on the repository, I have tried to honestly use content that is self made, open source or explicitly open for fair use, and citations should be there. If however you are the copyright holder and you want the material taken down, please flag up the issue accordingly and I will happily try and swap out the relevant material.

OCES 5303 :
ML methods in Ocean Sciences

Session 10: FNOs

Outline

- Fourier Neural Operators

  → the keyword is the **operator** part

  → quick recap/overview of Fourier analysis

  → structure of FNO

- FNO for three examples

  → 1d FNO for linear PDE

  → 1d FNO for nonlinear PDE

  → 2d FNO for image regression

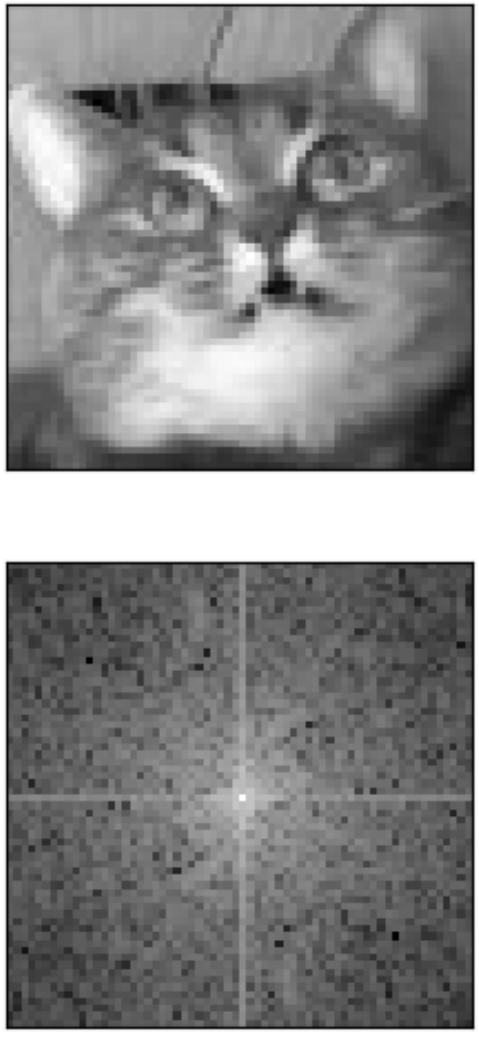


Figure: Head ad-hoc TA and her **power spectrum**.

Neural Networks vs. Neural **Operators**

- **neural networks** are intended to be maps between vectors that live in some **finite** dimensional data space

  $\rightarrow$ classification of cats/dogs: (64^2)d vector to 1d number

  $\rightarrow$ dimension reduction of cats/dogs: (64^2)d to 4d say

  $\rightarrow$ regression of cats/dogs: (64×32)d vector to (64×32)d vector

Neural Networks vs. Neural **Operators**

- **neural networks** are intended to be maps between vectors that live in some **finite** dimensional data space

  $\rightarrow$ classification of cats/dogs: (64^2)d vector to 1d number

  $\rightarrow$ dimension reduction of cats/dogs: (64^2)d to 4d say

  $\rightarrow$ regression of cats/dogs: (64×32)d vector to (64×32)d vector

- **operators** map between **infinite** dimension spaces

Neural Networks vs. Neural **Operators**

- **neural networks** are intended to be maps between vectors that live in some **finite** dimensional data space

  $\rightarrow$ classification of cats/dogs: (64^2)d vector to 1d number

  $\rightarrow$ dimension reduction of cats/dogs: (64^2)d to 4d say

  $\rightarrow$ regression of cats/dogs: (64×32)d vector to (64×32)d vector

- **operators** map between **infinite** dimension spaces

  $\rightarrow$ idea: learn the mapping between **spaces**, generalise better to unseen data?

Neural Networks vs. Neural **Operators**

- in practice we cannot deal with infinite dimension spaces directly

  $\rightarrow$ finite ones we can represent as tensors/matrices/arrays

Neural Networks vs. Neural **Operators**

- in practice we cannot deal with infinite dimension spaces directly

  $\rightarrow$ finite ones we can represent as tensors/matrices/arrays

  $\rightarrow$ idea: take a "good" finite representation of the underlying space, and learn operations acting on that representation

Neural Networks vs. Neural **Operators**

- in practice we cannot deal with infinite dimension spaces directly

  $\rightarrow$ finite ones we can represent as tensors/matrices/arrays

  $\rightarrow$ idea: take a "good" finite representation of the underlying space, and learn operations acting on that representation

- Fourier representation is one of these

  $\rightarrow$ basically **sines** and **cosines**

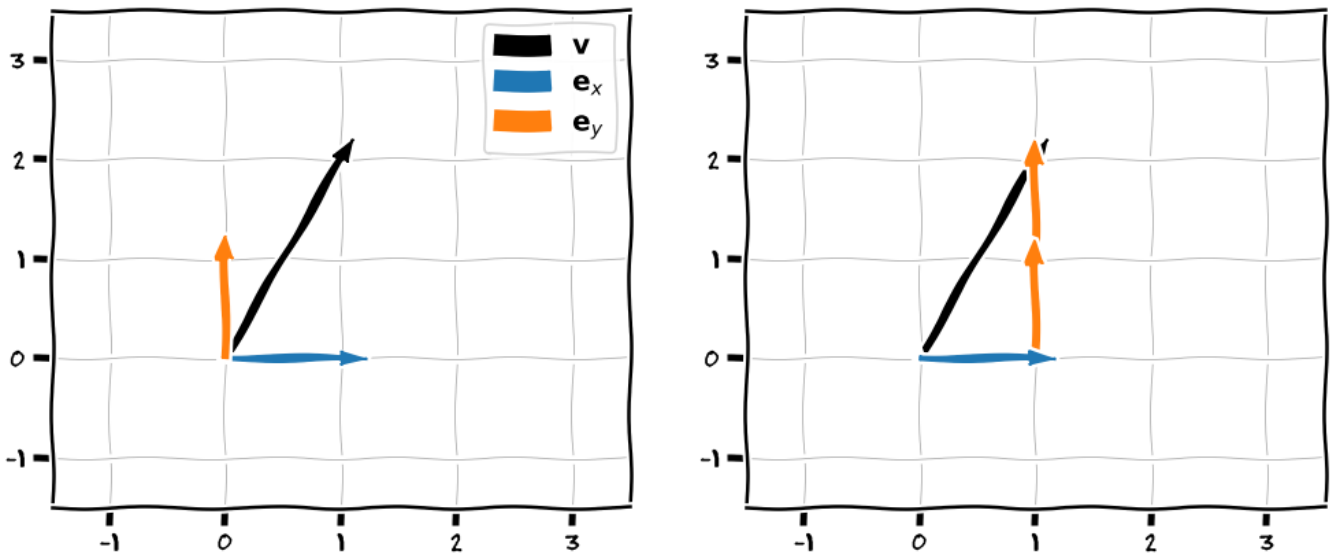
  $\rightarrow$ can represent a wide class of functions naturally or by artificial modifications

  $\rightarrow$ others entirely possible (e.g. spherical harmonics)

Recap: Fourier analysis

- standard basis in $\mathbb{R}^2$ is $e_1 = (1, 0)$ and $e_2 = (0, 1)$, so e.g.

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix} = e_1 + 2e_2$$



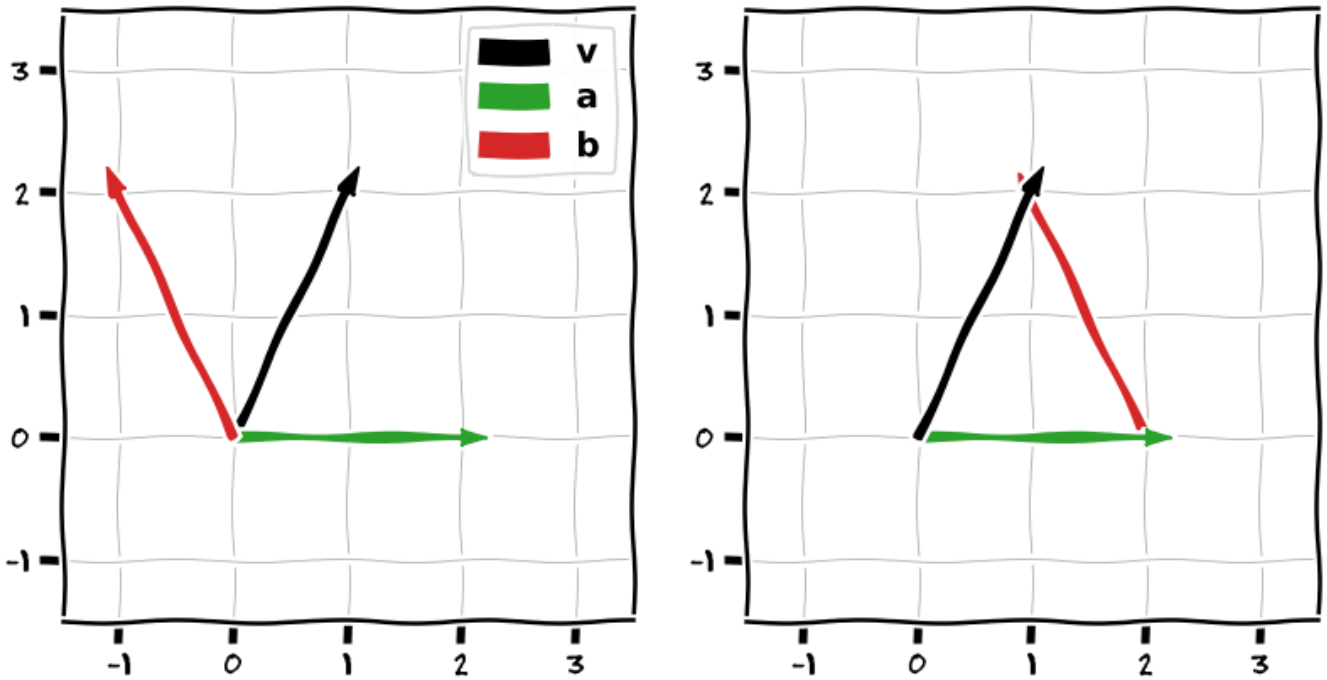
- $\{e_1, e_2\}$ form a **basis** that happens to be **orthogonal**

(with respect to the usual metric; actually ortho**normal** here)

Recap: Fourier analysis

- could do instead $\boldsymbol{a} = (2, 0)$ and $\boldsymbol{b} = (-1, 2)$, so e.g.

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix} = \boldsymbol{a} + \boldsymbol{b}$$



- $\{\boldsymbol{a}, \boldsymbol{b}\}$ is still a basis but not orthogonal
  $\rightarrow$ representation of $\boldsymbol{v}$ in this basis would be $(1, 1)$

Recap: Fourier analysis

- idea of a basis is quite general, e.g. color = (R, G, B)
  $\rightarrow$ cf. lego pieces for the larger structure

Recap: Fourier analysis

- idea of a basis is quite general, e.g. color = (R, G, B)

  $\rightarrow$ cf. lego pieces for the larger structure

- orthogonality is useful because then we have minimal overlaps between the co-ordinates/channels

  $\rightarrow$ cf. non-confounding variables

  $\rightarrow$ PCA/EOF looks for an orthogonal representation from data (the O in EOF)

would like an orthogonal basis for functions

Recap: Fourier analysis

- TL;DR: sines and cosines form an orthogonal basis for a wide class of functions, i.e.

$$f(x) = a_0 + \sum_{k=1}^{\infty} a_k \cos(kx) + b_k \sin(kx) = \text{Real}\left(\sum_{k=-\infty}^{\infty} c_k e^{ikx}\right)$$

- f has a **real** space representation of (for $f(x_i) = f_i$)

$$f(x) = [f_0, f_1, \ldots f_N]$$

and a Fourier/**spectral** representation of (imprecise with $N//2$ below)

$$\hat{f}(k) = [c_0, c_1, \ldots c_{\lfloor N/2 \rfloor}, c_{-\lfloor N/2 \rfloor}, \ldots c_{-1}]$$

Recap: Fourier analysis

- TL;DR: sines and cosines form an orthogonal basis for a wide class of functions, i.e.

$$f(x) = a_0 + \sum_{k=1}^{\infty} a_k \cos(kx) + b_k \sin(kx) = \text{Real}\left(\sum_{k=-\infty}^{\infty} c_k e^{ikx}\right)$$

- f has a **real** space representation of (for $f(x_i) = f_i$)

$$f(x) = [f_0, f_1, \ldots f_N]$$

and a Fourier/**spectral** representation of (imprecise with $N//2$ below)

$$\hat{f}(k) = [c_0, c_1, \ldots c_{\lfloor N/2 \rfloor}, c_{-\lfloor N/2 \rfloor}, \ldots c_{-1}]$$

- really for L^2 periodic functions
  $\rightarrow$ can manipulate functions to force them to be L^2 periodic

(e.g. padding, tapering)

Recap: Fourier analysis

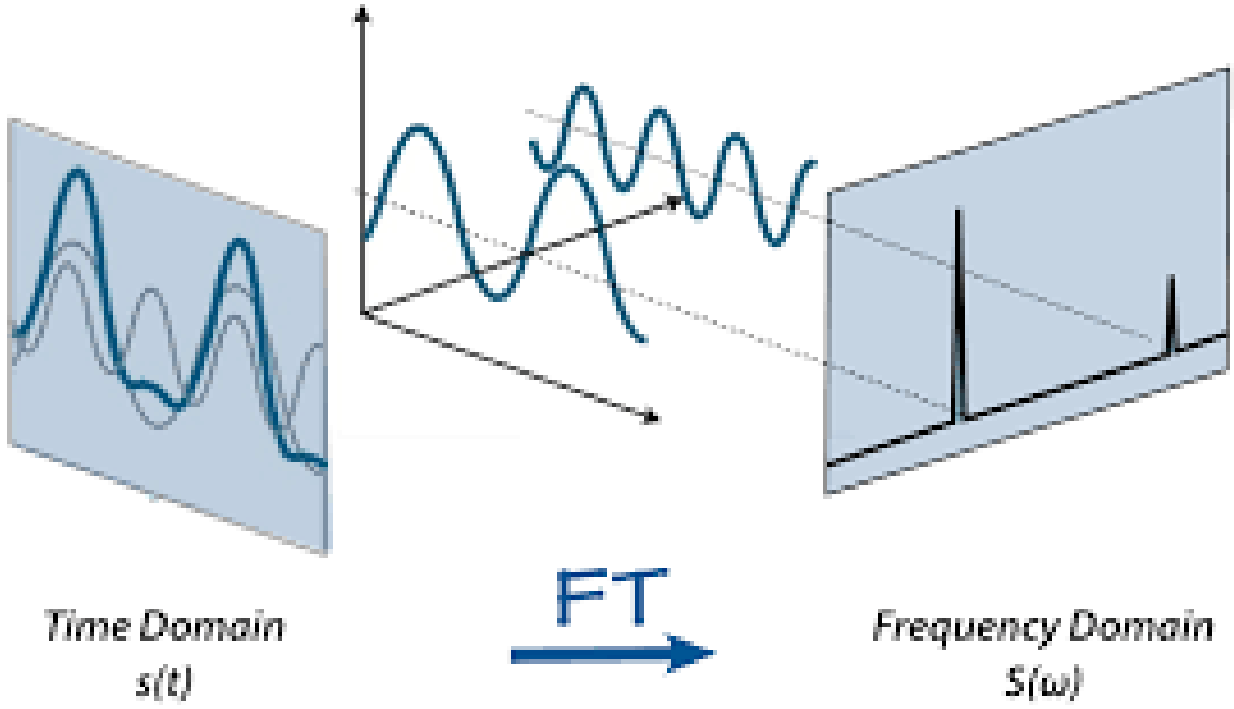


Figure: Schematic for the idea behind Fourier or spectral representation. From the internet (not sure where to attribute...)

- Fourier transform from real to spectral representation
  → e.g. signal analysis, imaging, etc.

Recap: Fourier analysis

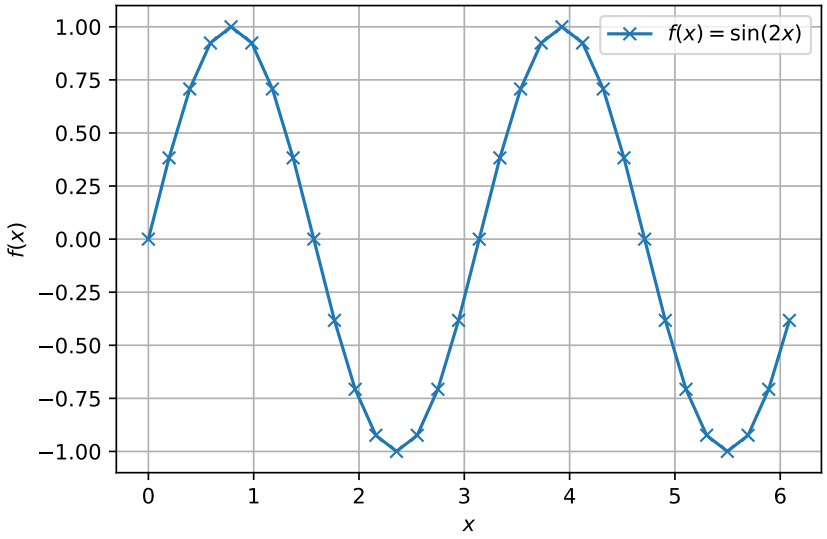


Figure: Example sine wave.

- take domain to be $[0, 2\pi]$ and $f(x) = \sin(2x)$

Q. what should the spectral representation look like?

$$f(x) = a_0 + \sum_{k=1}^{\infty} a_k \cos(kx) + b_k \sin(kx) = \text{Real}\left(\sum_{k=-\infty}^{\infty} c_k e^{ikx}\right)$$

Recap: Fourier analysis

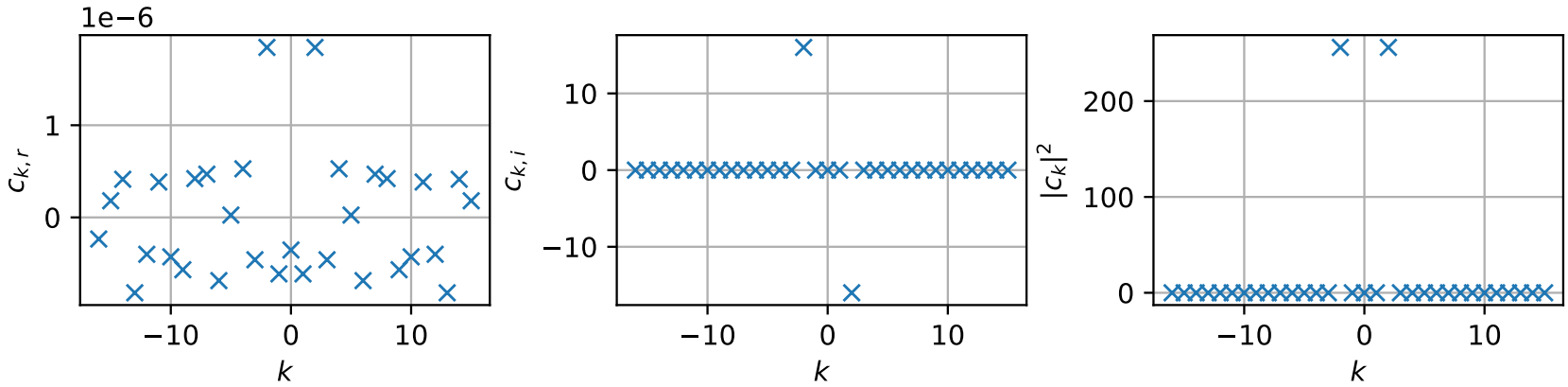


Figure: Fourier transform of the sine wave. (*a*) Real and (*b*) imaginary part of the Fourier coefficients, and (*c*) the power spectrum.

- done through `torch` (could use `numpy` too)

  $\rightarrow$ real components of c_k are tiny

  $\rightarrow$ non-zero imaginary components of c_k only at $k = 2$

  $\rightarrow$ non-zero power spectrum (i.e. $|c_k|^2$) only at $k = 2$

Recap: Fourier analysis

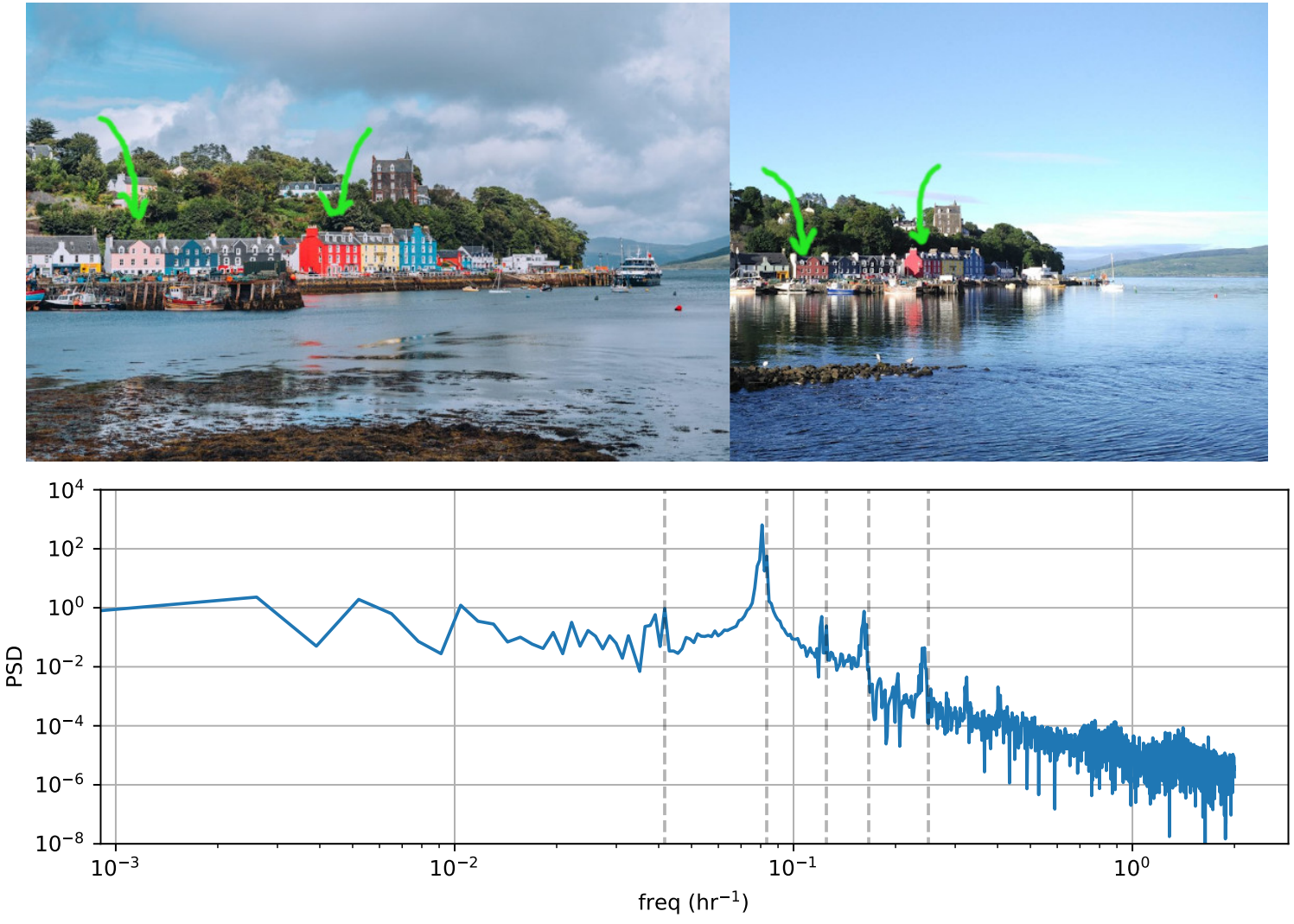


Figure: High and low tide at Tobermory, and its power spectrum computed from tide gauge data.

Recap: Fourier analysis

- can manipulate Fourier coefficients to do various operations
  $\rightarrow$ transform to spectral space, manipulate, transform back to real space

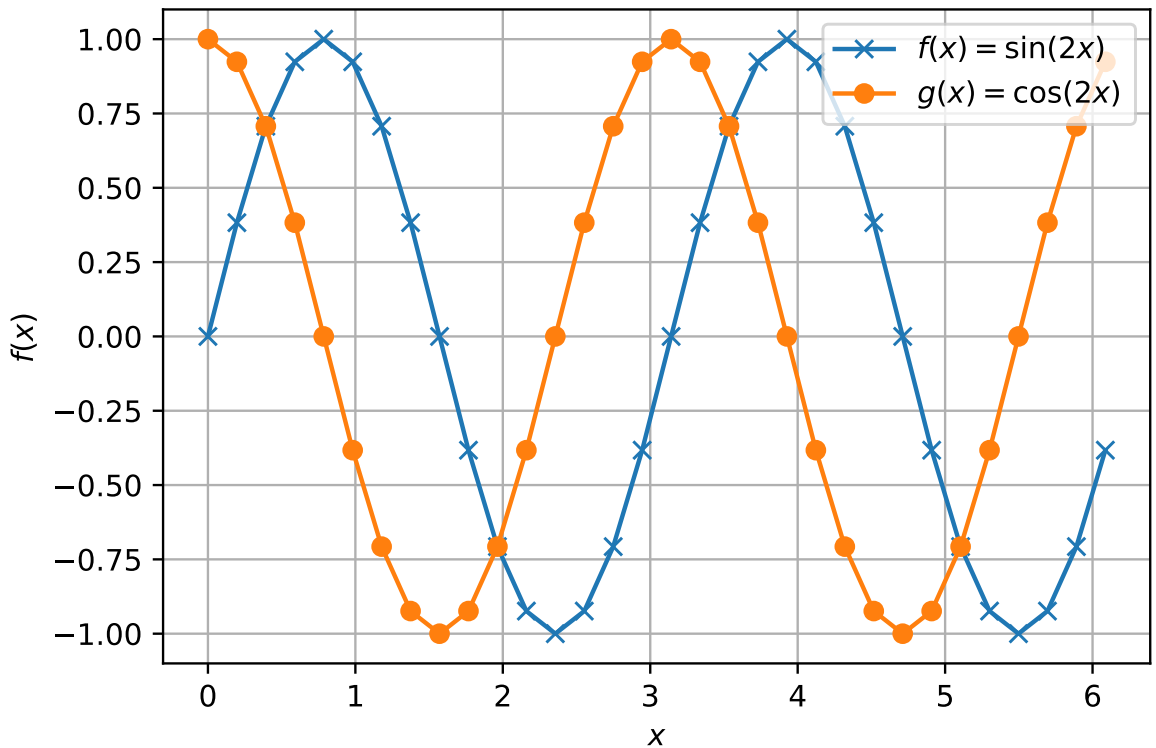


Figure: Example result from manipulating the Fourier coefficients in spectral space: shifting a wave.

Recap: Fourier analysis

- multiply all c_k by ik and $(ik)^2 = -k^2$

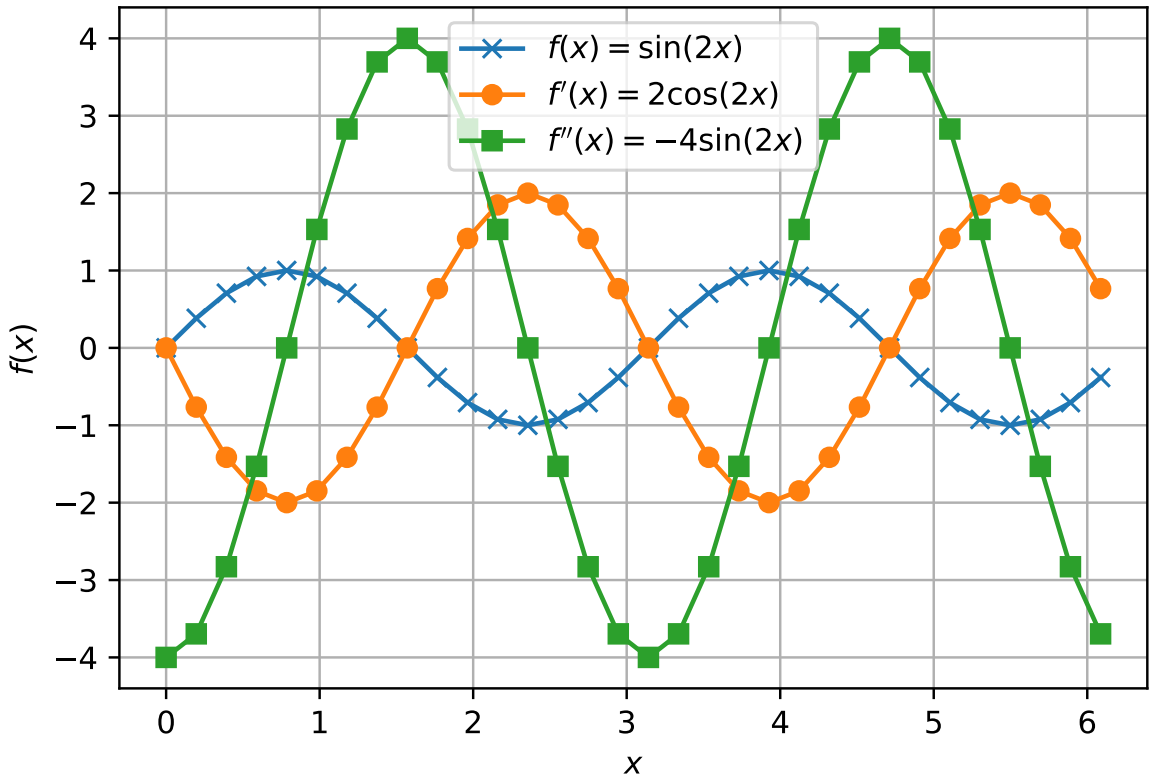


Figure: Example result from manipulating the Fourier coefficients in spectral space: taking derivatives.

Recap: Fourier analysis

- other manipulations possible, but main one of interest comes from the spectral convolution theorem:

convolution in one space

$\equiv$

pointwise multiplication in another

Recap: Fourier analysis

- other manipulations possible, but main one of interest comes from the spectral convolution theorem:

convolution in one space

$\equiv$

pointwise multiplication in another

- can learn convolutions in real space by learning multiplication in spectral space

  $\rightarrow$ FNOs like CNNs but with domain-wide kernel

  $\rightarrow$ can use similar machinery as before, just need to transform the data into spectral space

FNO

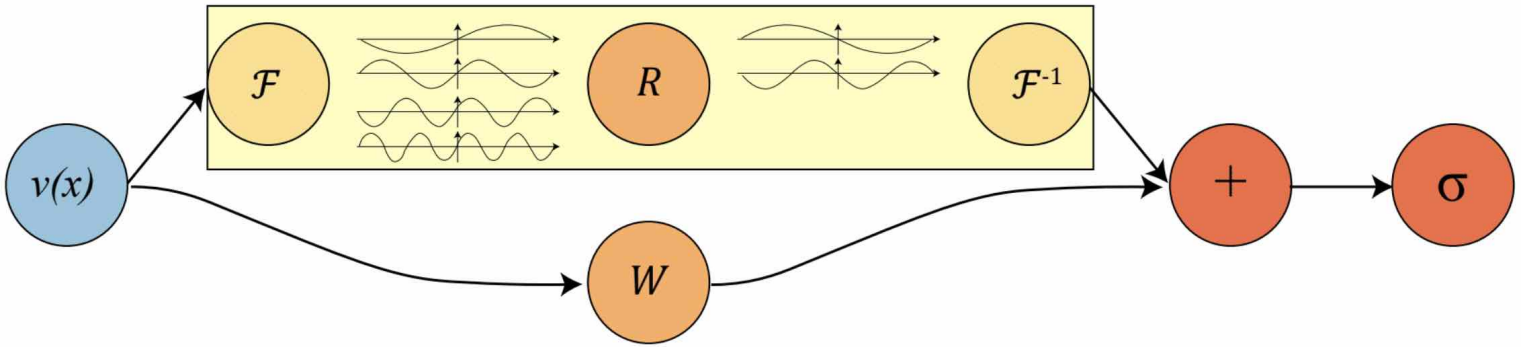


Figure: Schematic for the FNO block. From Fig. 2 of Li et al (2021).

- **lifting layer** $v(x)$: expands the number of channels
- **Fourier layer** $\mathcal{F}^{-1}R\mathcal{F}$: transform ($\mathcal{F}$), learn what to multiply in spectral space ($\mathcal{R}$), transform back ($\mathcal{F}^{-1}$)
- **biases** W: convolution in real space (so multiplication in spectral space) for mode mixing
- add together $+$, then do activation σ

  $\rightarrow$ nonlinear activation function to do some **mode mixing**
- **project** back down at the end

FNO

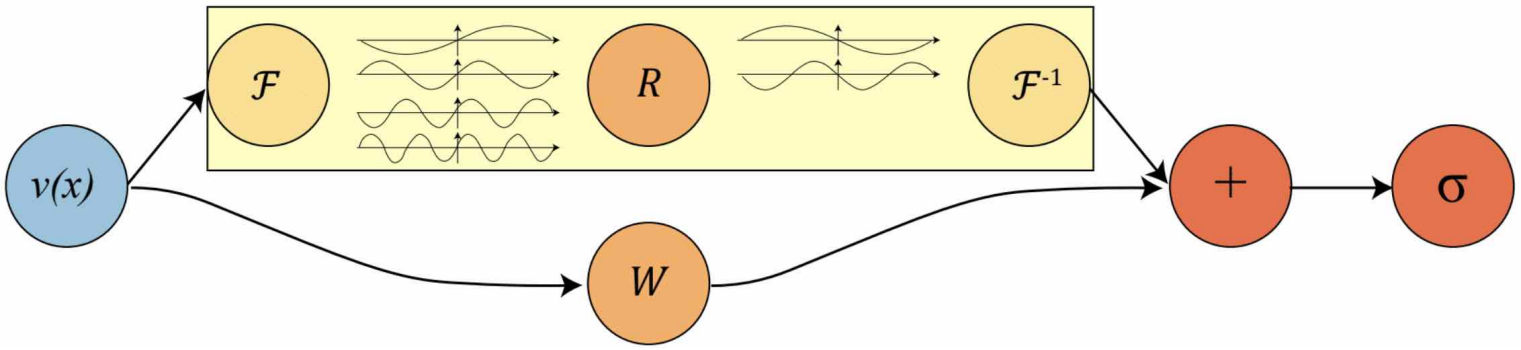


Figure: Schematic for the FNO block. From Fig. 2 of Li et al (2021).

- grid information concatenated into feature space

- non-periodicities?
  $\rightarrow$ can **pad** it (in third example)
- boundary conditions + nonlinearities?
  $\rightarrow$ biases and the activation
- generalisability?
  $\rightarrow$ should be **resolution** independent

FNO1d: linear PDE

- do diffusion equation again

  $\rightarrow$ input is $u(x, t = t_0)$, output is $u(x, t = t_0 + \Delta t)$

  $\rightarrow$ cf. RNN-type testing

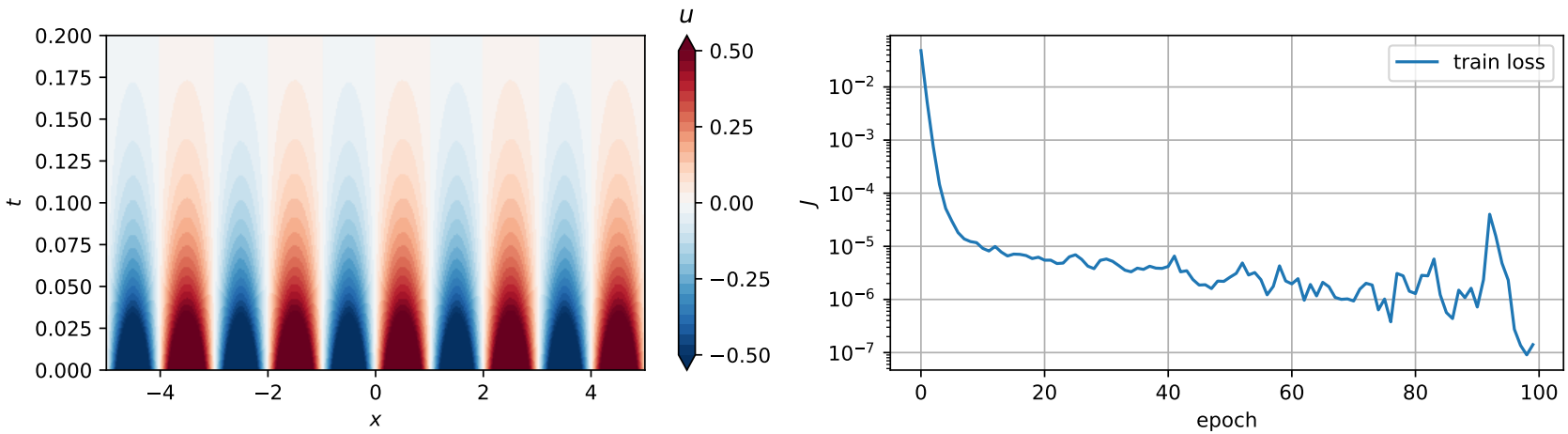


Figure: Data for a particular initialisation and choice of κ from 1d diffusion equation (periodic boundary condition here), and the loss from a simple `FNO1d`.

FNO1d: linear PDE

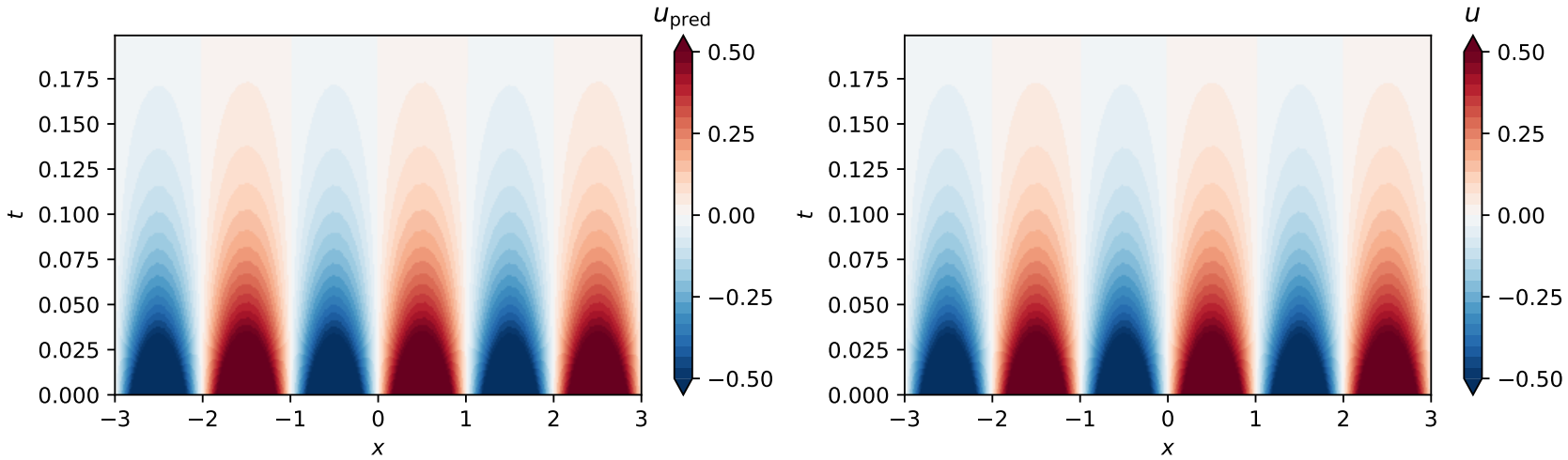


Figure: (Left) One-step prediction: $u_{\text{pred}}(t = t_0 + \Delta t) = F(u(t_0))$. (Right) Model truth.

- one-step prediction: just predict one time-frame from another

  $\rightarrow$ pretty good, as it should be, since that's how we trained it in the first place

FNO1d: linear PDE

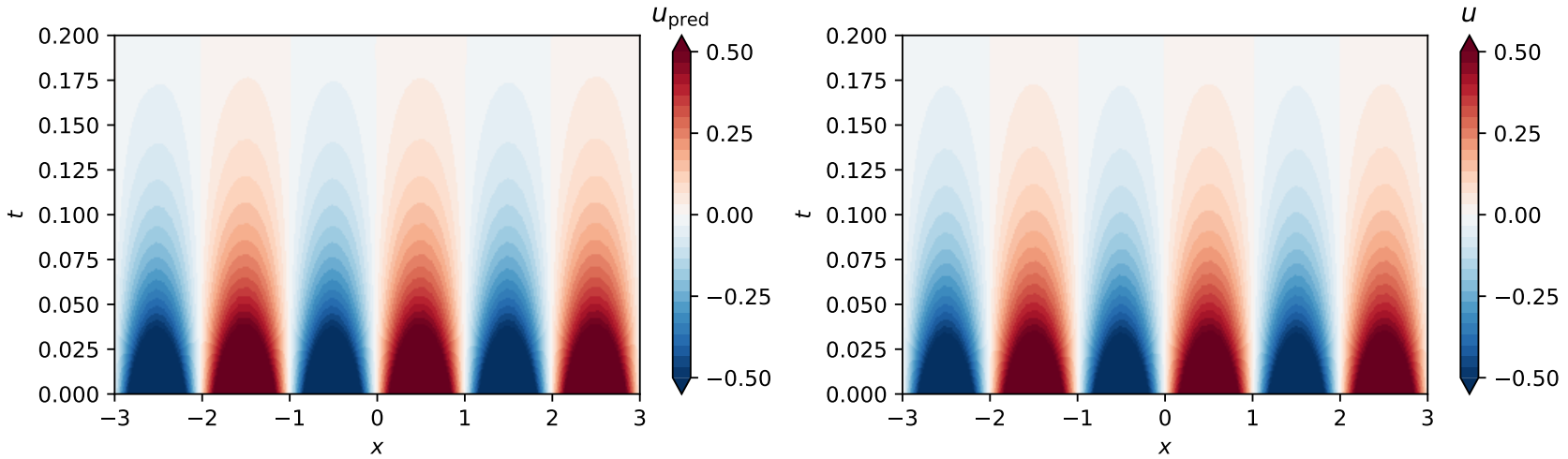


Figure: (Left) Sequential prediction: $u_{\text{pred}}(t = t_0 + n\Delta t) = F^n(u(t_0))$. (Right) Model truth.

- sequential prediction: make one prediction, pass that to model for another prediction and continue

  $\rightarrow$ much harder test

  $\rightarrow$ actually seems ok also!

FNO1d: **non**linear PDE

- do KdV again

  $\rightarrow$ input is $u(x, t = t_0)$, output is $u(x, t = t_0 + \Delta t)$

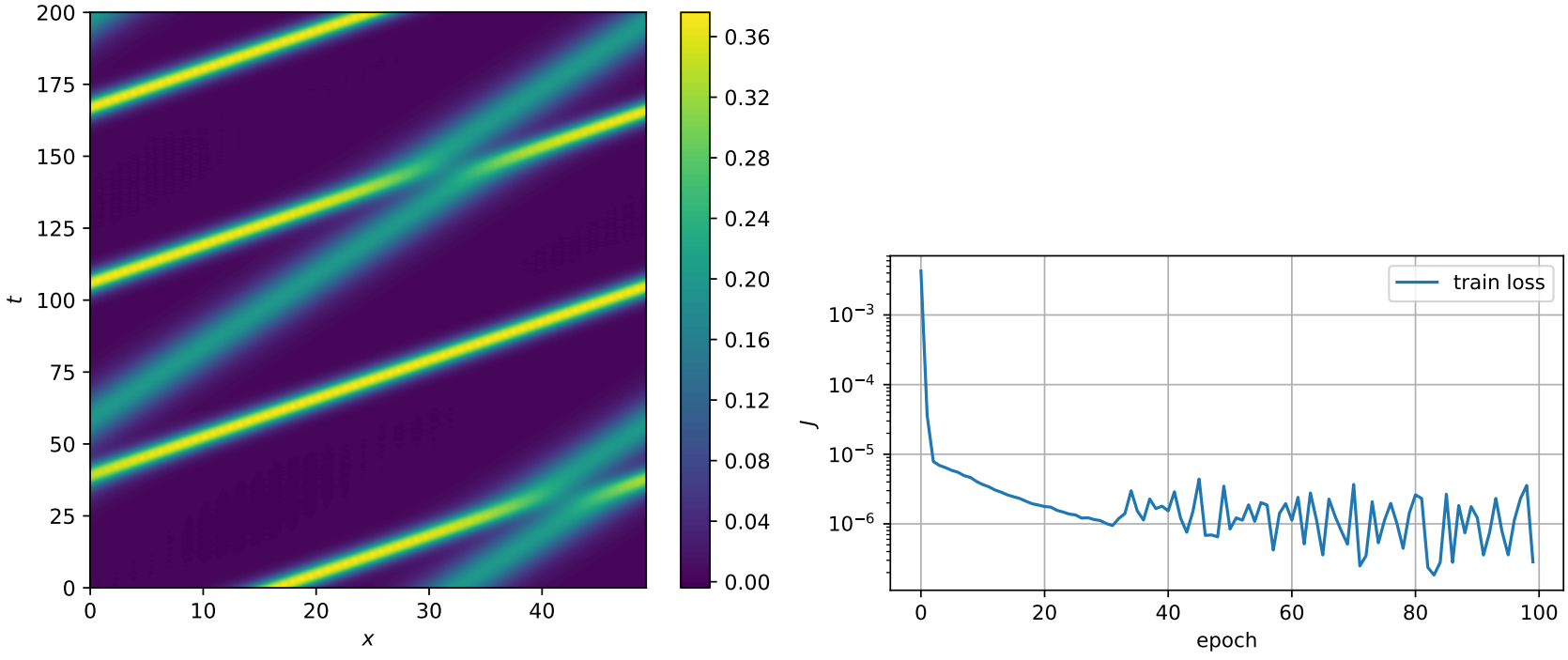


Figure: Data for a particular initialisation from KdV equation (periodic boundary condition here), and the loss from a simple `FNO1d`.

FNO1d: **non**linear PDE

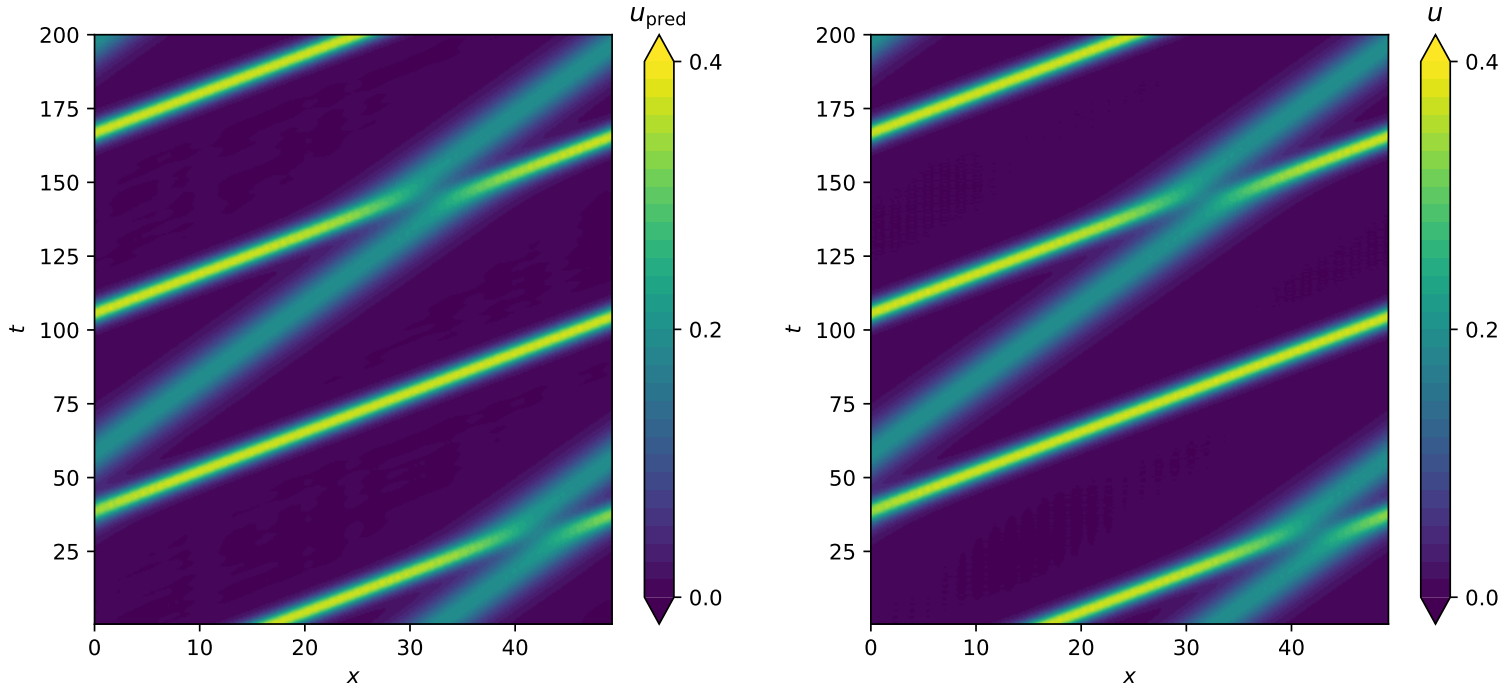


Figure: (Left) One-step prediction: $u_{\text{pred}}(t = t_0 + \Delta t) = F(u(t_0))$. (Right) Model truth.

- one-step prediction: pretty good as it should be

FNO1d: **non**linear PDE

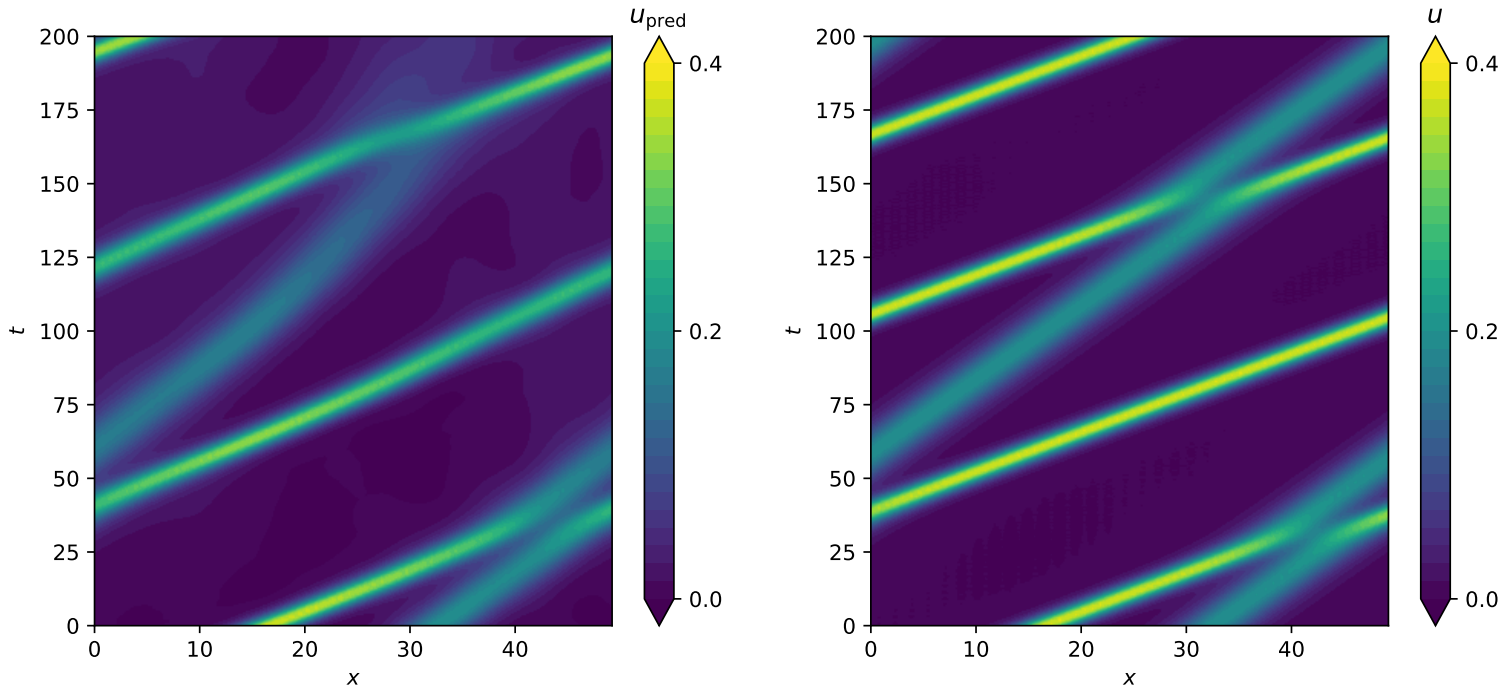


Figure: (Left) Sequential prediction: $u_{\text{pred}}(t = t_0 + n\Delta t) = F^n(u(t_0))$. (Right) Model truth.

- sequential prediction: not so good, but has potential?

FNO2d: image regression

- FNOs restricted to only differential equations and data with periodic boundary conditions?

FNO2d: image regression

- FNOs restricted to only differential equations and data with periodic boundary conditions?

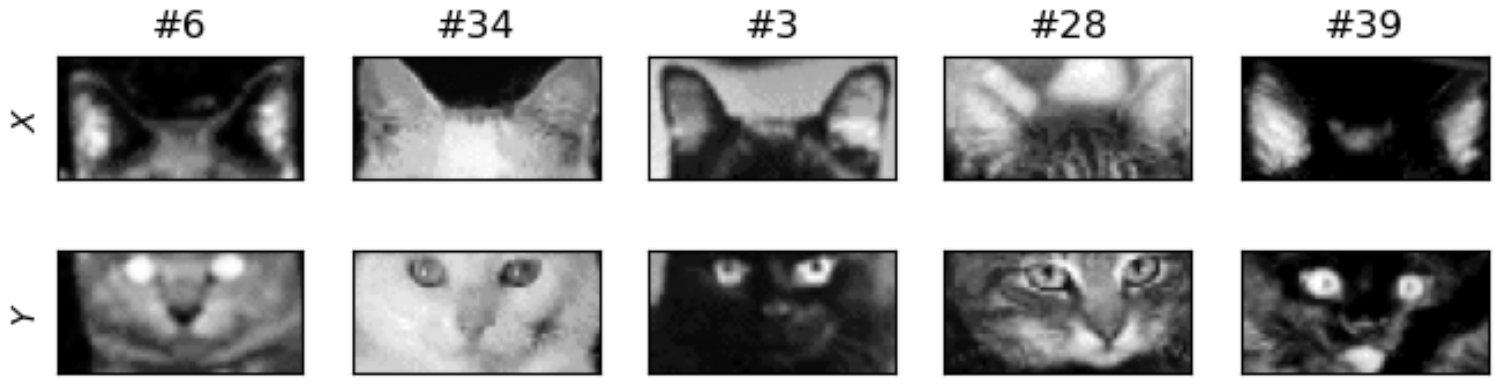


Figure: Unwitting subjects of abuse.

- structure essentially the same, with details to for 2d data

  $\rightarrow$ adds **padding** to data to formally deal with non-periodic data

FNO2d: image regression

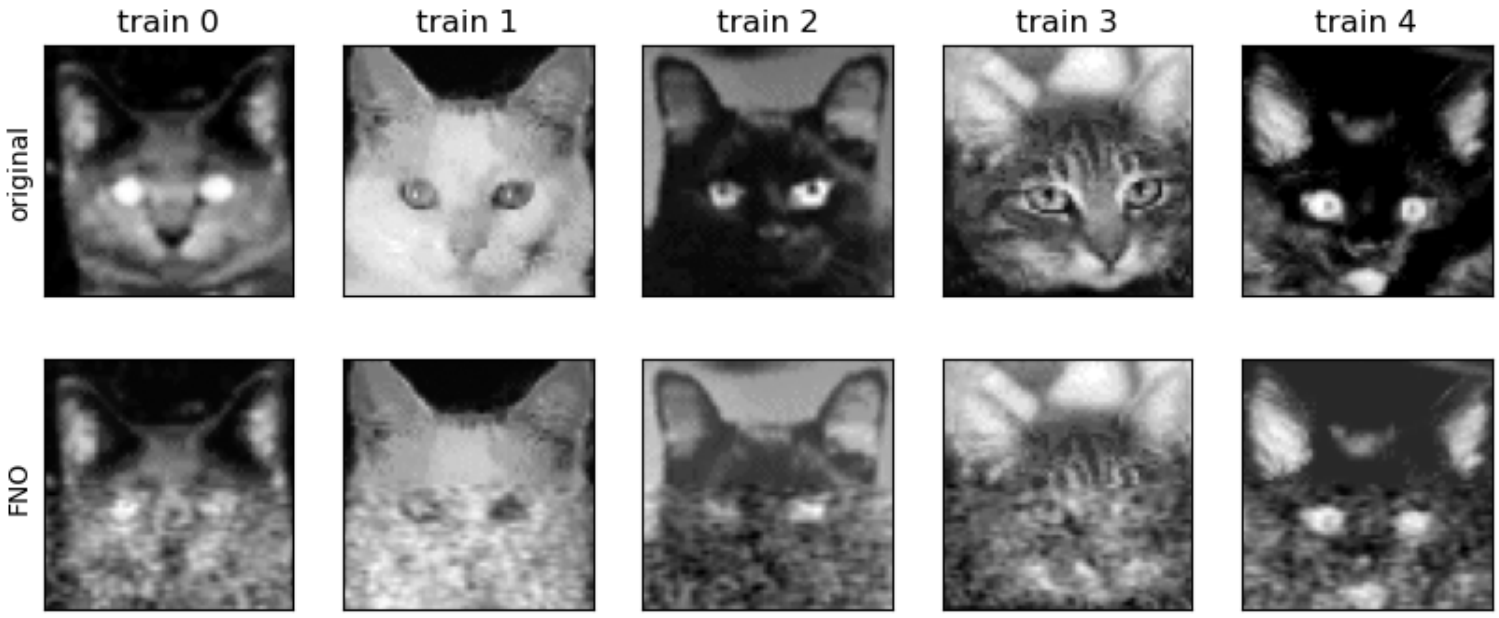


Figure: Unwitting subjects of abuse.

- ...has potential?

FNO2d: image regression

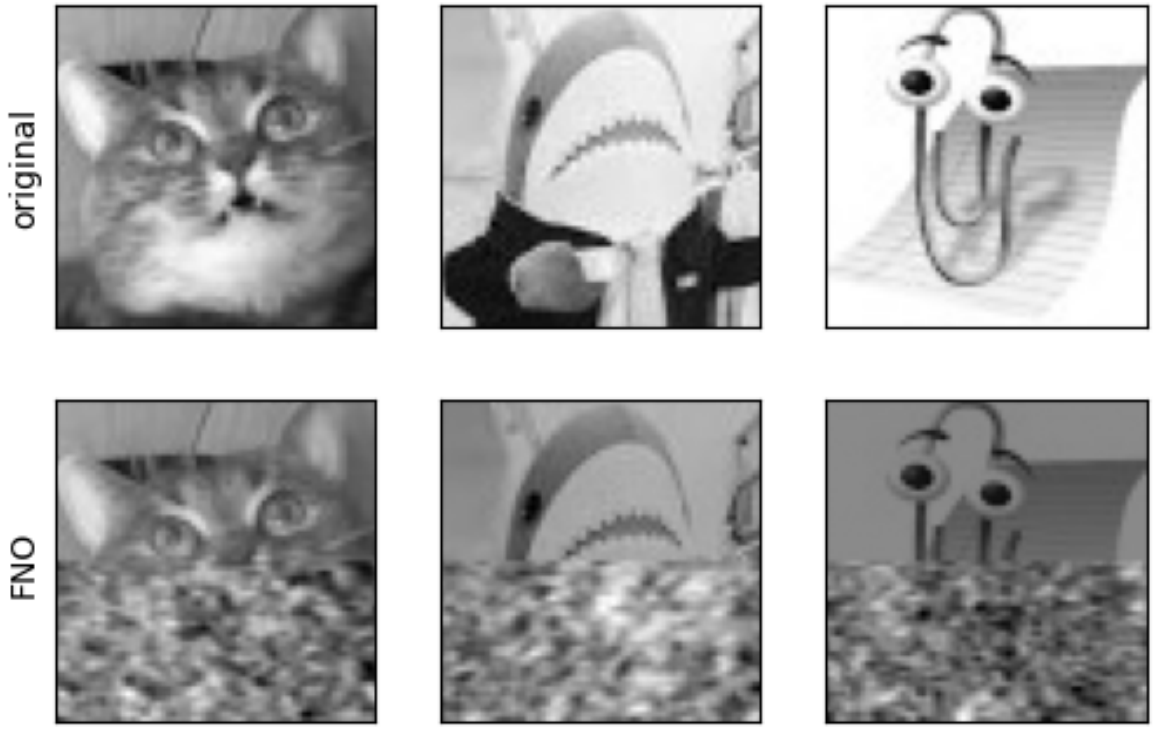


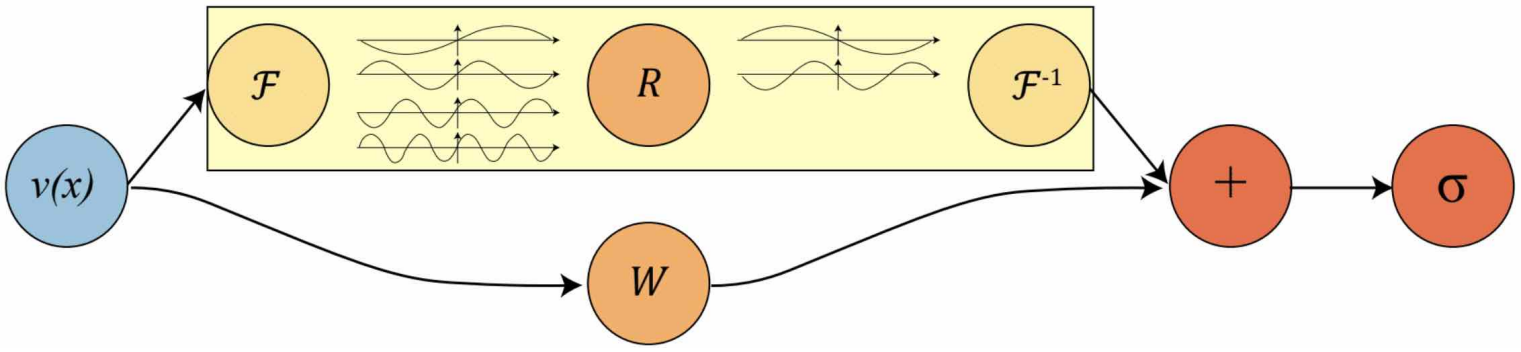
Figure: Further unwitting subjects of abuse.

- just noise here really

  $\rightarrow$ I didn't make the model too complex

  $\rightarrow$ this is a hard problem

Demonstration



- introduced FNOs

  → main keyword is **operator**, the **Fourier** is one realisation

  → probably most suited to differential equations, but can in principle be used for other things too
- variants thereof (e.g. RNOs, PINOs, etc.)

  → there is also the `NeuralOperators` package that I didn't use here (but you should try it)

Example variant: PINO

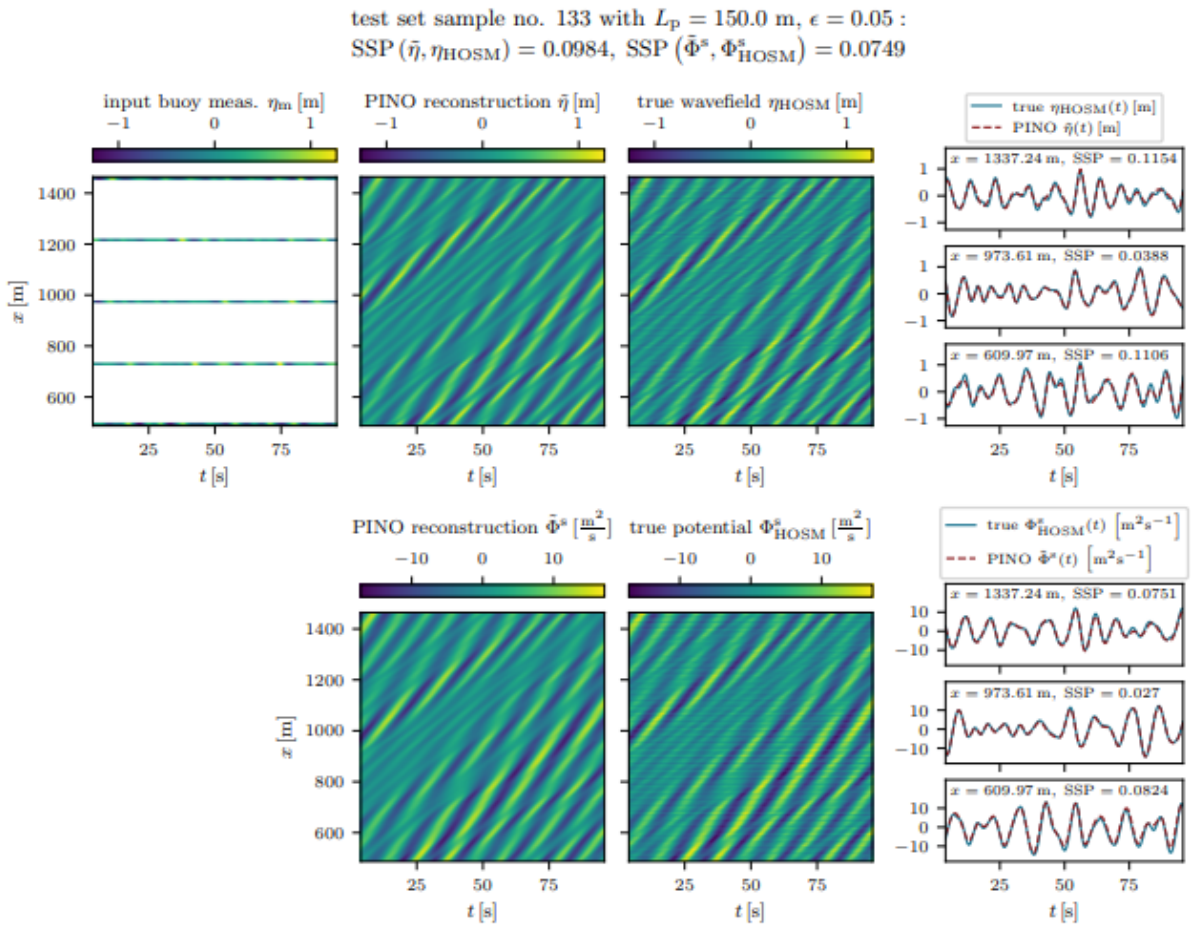


Figure: Reconstructing nonlinear waves with PINO. From Ehlers et al. (2025), Fig. 3.

Moving to a Jupyter notebook →